

Cinema-Reservation microservices system

Alessio Fior

Università degli studi di Roma Tor Vergata

Matricola: 0381477

Abstract—This project presents the design and implementation of a distributed cinema reservation platform based on a microservices architecture. The system addresses the challenges of scalability, consistency, concurrency management, and fault tolerance in highly concurrent booking scenarios, where multiple users may attempt to reserve the same seat simultaneously. Traditional monolithic architectures often struggle to guarantee responsiveness and reliability under concurrent workloads, motivating the adoption of loosely coupled distributed services.

The proposed solution is composed of multiple independent microservices, including an API Gateway, a Catalog Service, a Booking Service, a Notification Service, and a Recommendation Service. Inter-service communication is implemented using gRPC to provide efficient and strongly typed remote procedure calls, while asynchronous event propagation is handled through NATS JetStream following an event-driven architecture. Redis is employed both as a distributed synchronization mechanism and as a lightweight in-memory datastore for shared state management. All services are containerized and orchestrated using Docker Compose to simplify deployment and scalability.

Particular attention is devoted to concurrency control and system resilience. The Booking Service integrates distributed locking through Redis in order to avoid double-booking conditions during simultaneous seat reservation attempts. Furthermore, an Event Sourcing approach is adopted to reconstruct booking states from persisted events published on the messaging broker. To improve fault tolerance and prevent cascading failures, the API Gateway implements the Circuit Breaker pattern using Gbreaker, enabling graceful degradation when backend services become unavailable or overloaded.

I. INTRODUCTION

Modern reservation platforms must guarantee consistency, scalability, and fault tolerance while handling concurrent user requests. This project presents a distributed cinema reservation system based on a microservices architecture, where each service is responsible for a specific business capability such as movie catalog management, seat booking, notifications, and recommendations.

To achieve these capabilities, the system decouples core movie information from actual scheduled events through a clear separation between Movie and Projection entities. The Movie entity acts as a static catalog entry for metadata, while the Projection entity represents a specific instance of a screening, mapping a movie to a physical room and a temporal coordinate. This normalized structure ensures high scalability, avoiding data redundancy and allowing independent management of theater capacities and schedules.

The platform allows users to browse movies, visualize projections and seat availability, reserve seats in real time, and receive personalized recommendations. The system combines

several distributed systems technologies and patterns, including synchronous communication through gRPC, asynchronous event-driven communication with NATS JetStream, distributed locking with Redis, Event Sourcing for state recovery, and resilience mechanisms such as the Circuit Breaker pattern.

A primary objective of the project is ensuring consistency during concurrent booking operations, preventing race conditions and double reservations. At the same time, the architecture is designed to remain resilient to partial failures and easily scalable through service replication and containerization with Docker Compose. The entire platform was deployed on an Amazon EC2 virtual machine, allowing the system to be executed in a realistic cloud environment and enabling distributed deployment of the various microservices and infrastructure components.

Overall, the project represents a practical case study for applying modern cloud-native and distributed system techniques to the development of a reliable and scalable reservation platform.

II. BACKGROUND

This project is based on several concepts and technologies commonly adopted in modern distributed systems and cloud-native applications. In particular, the system combines microservices architecture, synchronous and asynchronous communication paradigms, distributed synchronization mechanisms, and resilience patterns to achieve scalability and consistency in concurrent reservation scenarios.

A. Microservices Architecture

Microservices architecture is a software design paradigm in which an application is decomposed into multiple small and independent services, each responsible for a specific business capability. Unlike monolithic systems, where all functionalities are tightly integrated into a single deployable unit, microservices promote modularity, independent deployment, scalability, and maintainability.

Each service typically owns its internal logic and data, communicating with other services through lightweight network protocols. This separation allows individual services to scale independently according to workload requirements and simplifies the evolution of the system over time.

In this project, the platform is divided into multiple services including:

- **API Gateway**
- **Catalog Service**

- **Booking Service**
- **Notification Service**
- **Recommendation Service**

This decomposition enables clear separation of concerns and allows asynchronous workflows to be implemented efficiently.

B. gRPC and Remote Procedure Calls

gRPC [1] is a high-performance Remote Procedure Call (RPC) framework developed by Google. It is based on Protocol Buffers (Protobuf), a language-neutral serialization mechanism that enables compact and efficient binary communication between distributed services.

Compared to traditional REST APIs, gRPC offers several advantages:

- Lower latency due to binary serialization
- Strongly typed service contracts through Protobuf definitions
- Efficient bidirectional communication over HTTP/2
- Automatic generation of client and server code

In this system, gRPC is used for synchronous communication between services. For example, the API Gateway communicates with the Catalog and Booking services through gRPC calls in order to retrieve movie information and process reservations.

Despite its performance benefits, gRPC also introduces tighter coupling between services because interfaces must be explicitly shared and synchronized through Protobuf schemas.

C. Event-Driven Architecture

An Event-Driven Architecture (EDA) is a distributed design approach in which services communicate asynchronously by producing and consuming events. Instead of directly invoking another service, a component publishes an event describing a state change, while interested services subscribe to and process such events independently.

This model improves:

- Loose coupling between services
- Scalability
- Fault isolation
- Asynchronous processing capabilities

The project adopts NATS JetStream as the messaging infrastructure. When a reservation is completed, the Booking Service publishes an event on a dedicated subject. Other services, such as the Notification Service and Recommendation Service, asynchronously consume these events to perform additional tasks without impacting the booking response time.

D. Event Sourcing

Event Sourcing [5] is a persistence pattern in which application state is reconstructed from a sequence of immutable events rather than being stored directly as mutable records.

Instead of storing only the current seat status, the system stores booking events.

The Booking Service can replay all stored events during startup in order to reconstruct the current room state. This mechanism provides several benefits:

- Improved auditability
- Recovery after service failures
- Historical traceability of operations
- Decoupling between state and persistence

NATS JetStream persistence capabilities are leveraged to implement this recovery mechanism.

E. NATS and JetStream Messaging

NATS [2] is a lightweight and high-performance messaging system designed for distributed and cloud-native applications. In this project, it was adopted as the backbone of the event-driven architecture to enable asynchronous communication between microservices.

Compared to other publish/subscribe platforms such as Apache Kafka, NATS was preferred because of its simplicity, lower resource consumption, and easier deployment. Since the goal of the project was to build a lightweight distributed reservation platform rather than a large-scale data streaming infrastructure, NATS represented a more suitable solution.

The platform specifically uses **NATS JetStream**, an extension that adds persistence, acknowledgments, and durable consumers. Whenever a reservation is completed, the Booking Service publishes an event on a dedicated subject. Other services, such as the Notification Service and Recommendation Service, subscribe to these events and process them independently.

Although event-driven systems introduce additional complexity compared to synchronous communication, NATS provides an efficient compromise between simplicity, scalability, and reliability, making it particularly suitable for real-time distributed systems such as reservation platforms.

F. Redis and Distributed Locking

Redis [3] is an in-memory key-value datastore widely used for caching, synchronization, and distributed coordination. In this project, Redis is employed to implement distributed locks for seat reservation operations.

Concurrent reservation requests may generate race conditions if multiple users attempt to reserve the same seat simultaneously. To avoid inconsistent states, the Booking Service uses the SETNX operation with expiration time to acquire temporary locks on seats before confirming reservations.

This mechanism guarantees that only one process can reserve a specific seat at a given moment, preventing double-booking scenarios in distributed deployments.

Redis is also used by the Recommendation Service to maintain lightweight user preference scores generated from booking events.

G. Circuit Breaker Pattern

Distributed systems are vulnerable to cascading failures when overloaded or unavailable services continue receiving requests. The Circuit Breaker pattern addresses this issue by temporarily interrupting requests toward failing services.

The project implements this mechanism within the API Gateway using the Gobreaker library. After a configurable

number of consecutive failures, the gateway opens the circuit and immediately rejects new booking requests for a limited amount of time.

This strategy improves:

- System resilience
- Failure isolation
- Recovery stability
- User experience under degraded conditions

The Circuit Breaker prevents excessive resource consumption and avoids propagating failures across the distributed system.

H. Containerization with Docker

All services are containerized using Docker and orchestrated through Docker Compose. Containerization guarantees environment consistency, simplifies deployment, and facilitates horizontal scaling of services.

Each microservice runs inside an isolated container together with the required infrastructure components, including Redis and NATS. Docker Compose provides service discovery and networking capabilities, enabling services to communicate transparently inside the distributed environment.

III. SOLUTION DESIGN

A. System Architecture

The proposed platform follows a distributed microservices architecture in which each service is responsible for a specific business capability. The system is composed of five main services and two infrastructure components:

- **API Gateway**
- **Catalog Service**
- **Booking Service**
- **Notification Service**
- **Recommendation Service**
- **Redis**
- **NATS JetStream**

The architecture combines both synchronous and asynchronous communication paradigms. User-facing operations requiring immediate responses are implemented through synchronous gRPC communication, while background processing and inter-service event propagation are handled asynchronously through NATS JetStream.

Figure 1 illustrates the high-level architecture of the platform.

B. API Gateway

The API Gateway represents the single entry point for client applications. It exposes REST endpoints to the external world and internally translates HTTP requests into gRPC calls toward backend services.

The gateway is responsible for:

- Request routing
- Response aggregation
- Error handling
- Circuit Breaker protection

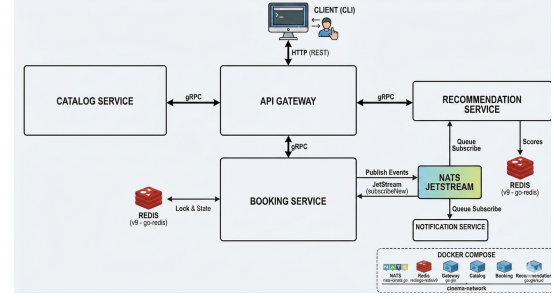


Fig. 1. High-level system architecture

This design hides internal service complexity from clients and centralizes communication management. The gateway also improves maintainability because backend services can evolve independently without exposing implementation details externally.

C. Catalog Service

The Catalog Service manages static information related to movies and projections. It provides functionalities such as:

- Retrieving movie lists
- Fetching movie details
- Retrieving available projections
- Querying projections associated with a specific movie

The service stores movie metadata loaded from a JSON configuration file and exposes gRPC endpoints consumed by both the API Gateway and the Recommendation Service.

Since the catalog data is mostly read-only, the service remains stateless and lightweight.

D. Booking Service

The Booking Service represents the core transactional component of the system. It manages seat availability and reservation requests while guaranteeing consistency under concurrent access.

The service maintains the occupancy state of each projection and coordinates reservation operations using:

- Local synchronization primitives (Mutex)
- Distributed locks through Redis
- Event persistence through NATS JetStream

Whenever a booking request is received:

- 1) The service validates the requested seat.
- 2) A distributed Redis lock is acquired.
- 3) Seat consistency checks are performed.
- 4) A booking event is published to NATS.
- 5) The local room state is updated.

The service also reconstructs room state during startup by replaying historical booking events stored in JetStream, implementing an Event Sourcing mechanism.

E. Notification Service

The Notification Service asynchronously processes booking events in order to simulate email confirmation delivery.

Instead of being directly invoked by the Booking Service, the component subscribes to booking events published on NATS subjects. This design introduces loose coupling and prevents notification latency from affecting booking performance.

The service uses queue subscriptions, enabling multiple instances to process notifications collaboratively in scalable deployments.

F. Recommendation Service

The Recommendation Service generates personalized movie recommendations based on users' booking history.

The service consumes booking events asynchronously and updates user preference scores stored in Redis. Preferences are inferred using:

- Movie categories
- Previously booked movies
- Projection time slots

When recommendation requests are received, the service retrieves movie metadata from the Catalog Service through gRPC and computes a ranking score for each movie according to the accumulated user preferences.

This architecture decouples recommendation logic from the booking workflow while enabling near real-time personalization updates.

G. Infrastructure Components

1) *Redis*: Redis acts as both a synchronization and lightweight persistence layer.

Within the Booking Service, Redis is used to implement distributed locks preventing concurrent double-bookings. Each seat reservation temporarily acquires a lock associated with the target seat.

Within the Recommendation Service, Redis stores user preference scores that are continuously updated from booking events.

2) *NATS JetStream*: NATS JetStream provides asynchronous messaging and durable event persistence.

The broker enables:

- Event propagation between services
- Event replay for state reconstruction
- Reliable message delivery
- Horizontal scalability through queue groups

Booking events are published using structured subjects such as:

```
bookings.event.<projection_id>
```

This naming strategy enables selective event subscriptions and efficient event organization.

H. Architectural Considerations

The chosen architecture introduces several advantages:

- Clear separation of concerns
- Independent scalability of services
- Improved fault isolation
- Flexible asynchronous integration
- Better maintainability

At the same time, distributed architectures also introduce additional operational complexity related to synchronization, monitoring, network communication, and infrastructure management. The project intentionally embraces these trade-offs in order to explore realistic distributed systems challenges.

IV. IMPLEMENTATION DETAILS AND ADOPTED DESIGN PATTERNS

This section describes the internal implementation of the platform and highlights the main software engineering and distributed systems patterns adopted throughout the project. Particular attention is devoted to concurrency management, resilience mechanisms, asynchronous communication, and extensibility strategies.

A. API Gateway and Circuit Breaker Pattern

The API Gateway acts as the external access point of the platform and is responsible for routing client requests toward backend services through gRPC communication.

One of the most important aspects of the gateway implementation is the adoption of the **Circuit Breaker** pattern. In distributed systems, repeated calls toward failing services may generate cascading failures and resource exhaustion. To prevent this issue, the gateway integrates the Gobreaker [4] library to monitor failures generated by the Booking Service.

The Circuit Breaker operates according to three states:

- **Closed State**: requests are forwarded normally.
- **Open State**: requests are immediately rejected after a configurable number of consecutive failures.
- **Half-Open State**: a limited number of requests are allowed to verify whether the service has recovered.

The following configuration is adopted:

- Maximum consecutive failures before opening the circuit: 3
- Recovery timeout: 15 seconds
- Maximum half-open requests: 5

When the circuit becomes open, the gateway immediately returns an error response without contacting the Booking Service:

```
"The booking system is temporarily overloaded.  
Please try again in a few seconds"
```

This mechanism improves resilience and protects backend services during overload conditions or temporary outages.

B. Booking Service and Concurrency Control

The Booking Service represents the core component of the platform because it manages concurrent seat reservations while guaranteeing consistency. The Booking Service is designed as a stateful microservice, meaning that it maintains the seat state in memory rather than relying on persistent storage. To scale this component across multiple replicas, the initial architectural idea was to adopt an Event Sourcing pattern, where state changes are stored as events inside the message broker and each service instance rebuilds its local state by replaying the event stream. However, the simpler and more

common approach would have been to design the Booking Service as a stateless microservice from the beginning, where all instances read and write state from a shared database or distributed cache. Nevertheless, due to the project requirements, the service was intentionally designed as stateful.

To prevent race conditions and double bookings, the service combines two synchronization mechanisms. At the distributed level, Redis locks are used through atomic `SETNX` operations, ensuring that only one service instance can reserve a specific seat at a given time. Each lock is associated with a unique token and a temporary expiration time (TTL) to avoid stale lock situations.

At the local level, the internal in-memory seat state is protected using Go `Mutex` primitives, preventing concurrent goroutines inside the same instance from modifying shared data simultaneously.

The combination of distributed locking and local mutual exclusion allows the platform to maintain strong consistency guarantees even under high concurrency scenarios while preserving scalability across multiple Booking Service replicas.

C. Event-Driven Communication

The platform adopts an **Event-Driven Architecture** for asynchronous communication between services.

Instead of directly invoking secondary services after a reservation, the Booking Service publishes booking events on NATS JetStream subjects:

```
bookings.event.<projection_id>
```

Other services independently subscribe to these events:

- Notification Service
- Recommendation Service

This architecture introduces several benefits:

- Loose coupling between services
- Independent scalability
- Reduced booking latency
- Improved fault isolation

For example, recommendation updates and notification delivery occur asynchronously without delaying the booking response returned to the client.

1) *Queue Groups*: The asynchronous consumers use NATS queue subscriptions:

```
QueueSubscribe(..., "recommendation_group",
```

Queue groups allow multiple instances of the same service to collaboratively consume events, enabling horizontal scalability and automatic load balancing.

D. Recommendation Service Logic

At runtime, the Recommendation Service consumes booking events from a NATS JetStream stream (`bookings.event.*`) using a durable queue subscription. Each event is deserialized into a `BookingEvent` containing user, movie, projection, and seat information. For every event, the service enriches the data by querying the Catalog Service to retrieve both projection and movie metadata. This allows

the system to extract meaningful signals such as movie category and show time.

These signals are then used to update a user-specific preference profile stored in Redis. The profile is represented as a hash (`user:{userID}:scores`) and updated using atomic increments through a Redis pipeline. Different behavioral signals are weighted differently: movie-level interactions receive the highest weight, followed by category affinity and time-slot preferences. This produces a continuously evolving implicit feedback model for each user.

When generating recommendations, the service retrieves the user's preference profile from Redis and fetches the full movie catalog from the Catalog Service. It then computes a simple scoring function in-memory by matching stored preference keys against each movie's attributes. Category matches contribute a baseline score, while direct movie interactions have a stronger impact through higher weighting. The final list of movies is sorted in descending order of score and returned to the user as a ranked recommendation list.

E. Event Sourcing Pattern

The Booking Service implements the **Event Sourcing** pattern through NATS JetStream persistence capabilities.

Instead of persisting only the latest seat state, the system stores immutable booking events representing reservation operations.

A typical event contains:

- Booking ID
- User ID
- Projection ID
- Seat ID

When the Booking Service starts, it reconstructs the current room state by replaying all historical booking events stored in JetStream:

```
sub.NextMsg(timeout)
```

This approach provides several advantages:

- Recovery after crashes or restarts
- Complete historical traceability
- Decoupling between persistence and state
- Improved auditability

Event replay enables the service to restore seat occupancy without relying on a traditional relational database.

F. Strategy Pattern for Seat Reservation Validation

The Booking Service adopts the **Strategy Pattern** [6] to encapsulate different seat validation policies in a modular and extensible way. This design pattern allows the system to vary the reservation rules dynamically without modifying the core booking logic.

Each strategy implements a common interface:

```
Validate(room *RoomState, seat int32) error
```

This ensures that all validation algorithms are interchangeable and can be selected at runtime based on configuration.

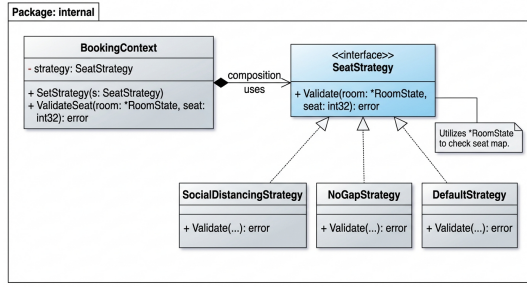


Fig. 2. strategy pattern

1) *Default Strategy*: The **Default Strategy** represents the simplest validation policy. It only checks whether a seat is already occupied:

```
if room.Seats[seat] {
    return error("seat already taken")
}
```

This strategy does not impose any additional constraints and is suitable for scenarios where seat adjacency rules are not required.

2) *Social Distancing Strategy*: The **Social Distancing Strategy** enforces a constraint that ensures at least one empty seat between two occupied seats. This is particularly relevant in scenarios where spacing between users is required.

The validation logic checks:

- Whether the selected seat is already occupied
- Whether adjacent seats (left and right) are free

If either neighbor seat is occupied, the reservation is rejected with a constraint violation.

This strategy ensures spatial separation between users in the same projection room.

3) *No Gap Strategy*: The **No Gap Strategy** aims to avoid isolated empty seats, optimizing seat occupancy distribution.

In addition to verifying that the selected seat is free, the strategy evaluates the surrounding configuration:

- It prevents leaving a single isolated empty seat between two occupied seats on the left side
- It applies the same constraint symmetrically on the right side

If such a configuration is detected, the reservation is rejected with an error such as:

"you would leave an isolated seat on the left/right"

This strategy improves space utilization by minimizing unusable isolated seats.

4) *Design Benefits*: The use of the Strategy Pattern provides several advantages:

- **Extensibility**: new reservation policies can be added without modifying existing code.
- **Separation of Concerns**: validation logic is isolated from booking logic.
- **Runtime Flexibility**: strategies can be selected dynamically based on configuration.

This design makes the Booking Service adaptable to different business rules and future domain requirements.

G. Notification Service

The Notification Service represents a lightweight asynchronous consumer responsible for processing booking confirmation events.

The service subscribes to booking subjects and simulates email delivery

The component demonstrates how event-driven systems simplify the integration of auxiliary functionalities without impacting transactional workflows.

V. RESULTS AND DISCUSSION

This section presents the experimental evaluation of the distributed cinema reservation platform. The analysis focuses on the main architectural properties of the system, including scalability, fault tolerance, asynchronous communication, and consistency under concurrent reservation requests.

Several tests were performed to evaluate the behavior of the platform during service failures, concurrent booking operations, and event-driven interactions through NATS JetStream. The obtained results demonstrate the effectiveness of the adopted distributed design patterns in ensuring reliability, scalability, and consistency.

A. Circuit Breaker Validation

In order to evaluate the resilience mechanisms implemented inside the API Gateway, a stress test was performed against the booking endpoint while the Booking Service was intentionally unavailable. The objective of the experiment was to verify the correct behavior of the Circuit Breaker pattern under repeated failure conditions.

The test consisted of repeatedly sending reservation requests through the following command:

```
for i in {1..5}; do
curl -X POST http://localhost:8080/book \
-H "Content-Type: application/json" \
-d '{"projection_id":"p101","seat_id":1,
"user_id":"test-user"}'
done
```

Booking-1	2026/05/09 08:20:14 [CB] failures=1 requests=1	172.21.0.1	POST	"/book"
getway-1	[GIN] 2026/05/09 08:20:14 500 441.80µs			
getway-1	2026/05/09 08:20:14 [CB] failures=2 requests=2	172.21.0.1	POST	"/book"
getway-1	[GIN] 2026/05/09 08:20:14 500 270.00µs			
getway-1	2026/05/09 08:20:14 [CB] failures=3 requests=3	172.21.0.1	POST	"/book"
getway-1	2026/05/09 08:20:14 [CB] closed -> open			
getway-1	[GIN] 2026/05/09 08:20:14 500 228.80µs	172.21.0.1	POST	"/book"
getway-1	[GIN] 2026/05/09 08:20:14 503 133.04µs	172.21.0.1	POST	"/book"

Fig. 3. Circuit Breaker activation during repeated booking failures

The observed behavior confirms the correct activation of the Circuit Breaker mechanism. During the first requests, the gateway attempted to contact the unavailable Booking Service, generating HTTP 500 responses corresponding to backend failures. The internal Gobreaker counters progressively increased the number of consecutive failures:

```
[CB] failures=2 requests=2
[CB] failures=3 requests=3
```


Once the configured threshold of three consecutive failures was reached, the Circuit Breaker transitioned from the closed state to the open state:

```
[CB] closed -> open
```

After the transition, subsequent requests were immediately rejected by the gateway with HTTP 503 responses, without forwarding traffic toward the Booking Service:

```
POST "/book" -> 503
```

The experiment demonstrates that the Circuit Breaker successfully prevents continuous propagation of failing requests toward unavailable backend services. This behavior is particularly important in distributed architectures because it avoids unnecessary resource consumption, reduces cascading failures, and improves recovery stability under overload or fault conditions.

The obtained results confirm that the adopted resilience strategy effectively protects the booking infrastructure and enables graceful degradation during partial system failures.

Note that for simplicity, Circuit Breaker mechanisms were not introduced for the Catalog Service and Recommendation Service. Both services are designed to be stateless and therefore can be easily replicated and scaled horizontally. In case of instance failures, the gRPC load balancing mechanism can transparently redirect requests toward other available replicas, ensuring service continuity without requiring additional recovery logic.

B. Concurrency and Consistency Validation

A concurrency stress test was performed in order to evaluate the correctness of the synchronization mechanisms implemented inside the Booking Service. The experiment simulated multiple users attempting to reserve the same seat simultaneously through concurrent HTTP requests.

The following command was executed:

```
for i in {1..20}; do
(
curl -s -X POST http://localhost:8080/book \
-H "Content-Type: application/json" \
-d "{
  \"projection_id\": \"p201\",
  \"seat_id\": 25,
  \"user_id\": \"user$i\"
}" &
)
done
```

Figure 4 shows the output generated during the execution of the test.

This behavior confirms the effectiveness of the combined synchronization strategy based on Redis distributed locks and local mutex protection. The distributed lock prevented multiple service instances from concurrently reserving the same seat, while local mutual exclusion guaranteed thread safety inside the service process itself.

```
booking-1 | 2026/05/09 08:15:07 Test
booking-1 | [GIN] 2026/05/09 - 08:15:07 | 200 | 40.67ms | 172.21.0.1 | POST "/book"
booking-1 | 2026/05/09 08:15:24 Test
booking-1 | [GIN] 2026/05/09 - 08:15:24 | 200 | 3.00ms | 172.21.0.1 | POST "/book"
notification-service-1 | 2026/05/09 08:15:27 [Notification] Ready, listening for new events...
notification-service-1 | 2026/05/09 08:15:37 [EMAIL SENT] Confirmation for user1: Seat 5 successfully booked!
notification-service-1 | 2026/05/09 08:15:37 [EMAIL SENT] Confirmation for user2: Seat 12 successfully booked!
```

Fig. 4. Concurrent booking requests targeting the same seat

C. Notification Service Recovery Test

The Notification Service was designed as a stateless asynchronous consumer, making it naturally scalable and resilient to temporary failures. In order to evaluate its recovery capabilities, a fault tolerance test was performed by intentionally stopping the service before generating a new booking event.

During the experiment, the Notification Service was intentionally stopped using the `docker stop` command, while the Booking Service continued to process reservation requests normally. Despite the temporary unavailability of the notification component, a new seat reservation was successfully completed while the consumer was offline.

After restarting the Notification Service using `docker start`, the system automatically recovered and processed the previously missed booking event stored in NATS JetStream. The pending notification was correctly consumed and delivered once the service became available again.

Figure 5 shows the execution of the experiment and the successful recovery of the pending notification event after the service restart.

```
notification-service-1 | 2026/05/09 08:15:07 Test
booking-1 | [GIN] 2026/05/09 - 08:15:07 | 200 | 40.67ms | 172.21.0.1 | POST "/book"
gateway-1 | [GIN] 2026/05/09 - 08:15:24 | 200 | 3.00ms | 172.21.0.1 | POST "/book"
notification-service-1 | 2026/05/09 08:15:27 [Notification] Ready, listening for new events...
notification-service-1 | 2026/05/09 08:15:37 [EMAIL SENT] Confirmation for user1: Seat 5 successfully booked!
notification-service-1 | 2026/05/09 08:15:37 [EMAIL SENT] Confirmation for user2: Seat 12 successfully booked!
```

Fig. 5. Notification Service recovery and delayed event processing

The experiment shows that NATS JetStream durable consumers and persistent streams prevent message loss during temporary service outages. Events remain stored inside the broker and can be processed once the consumer reconnects, ensuring reliability and consistency without affecting the main booking workflow.

D. Booking Service Replication and Event Sourcing Recovery

An additional experiment was conducted to evaluate the behavior of the replicated Booking Service under failure conditions. Multiple booking service instances were deployed simultaneously using command `docker compose up --scale booking=3`, while the API Gateway forwarded requests through gRPC load balancing.

During the test, several reservations were performed on different projections and seats. While the system was actively processing requests, individual Booking Service instances were intentionally terminated. Despite these failures, the platform continued processing reservations correctly because the

remaining instances were still available and reachable through the gateway.

Figure 6 shows the execution of the experiment and the behavior of the replicated booking infrastructure during instance failures.

```

booking-1 2020/06/09 11:35:44 INFO [EVENT PUBLIC] Subject: bookings.event.p001 | Data: {"booking_id":"RES-g001-1","user_id":"user1","projection_id":"p001","seat_id":1}
booking-1 2020/06/09 11:35:44 INFO [EVENT PUBLIC] Confirmation for user1: Seat 1 successfully booked!
booking-1 2020/06/09 11:35:44 INFO [EVENT PUBLIC] Subject: bookings.event.p002 | Data: {"booking_id":"RES-g001-2","user_id":"user2","projection_id":"p002","seat_id":2}
booking-1 2020/06/09 11:35:44 INFO [EVENT PUBLIC] Confirmation for user2: Seat 2 successfully booked!
booking-2 exited with code 0
booking-2 2020/06/09 11:36:29 Test
booking-2 2020/06/09 11:36:29 INFO [EVENT PUBLIC] Subject: bookings.event.p002 | Data: {"booking_id":"RES-g002-2","user_id":"user2","projection_id":"p002","seat_id":2}
booking-2 2020/06/09 11:36:29 INFO [EVENT PUBLIC] Confirmation for user2: Seat 2 successfully booked!
booking-2 exited with code 0
booking-2 exited with code 0
booking-2 exited with code 0
booking-2 2020/06/09 11:37:44 INFO [EVENT PUBLIC] Subject: bookings.event.p002 | Data: {"booking_id":"RES-g002-2","user_id":"user2","projection_id":"p002","seat_id":2}
booking-2 2020/06/09 11:37:44 INFO [EVENT PUBLIC] Confirmation for user2: Seat 2 successfully booked!
booking-2 exited with code 0
booking-2 exited with code 0
booking-2 exited with code 0

```

Fig. 6. Booking Service replication and recovery through Event Sourcing

The observed results confirm that service replication combined with gateway-side load balancing provides fault tolerance at the application level. When one instance became unavailable, incoming requests were transparently redirected toward the remaining active replicas without interrupting the booking workflow.

Furthermore, the Event Sourcing mechanism ensured state consistency across service restarts. Since all booking events were persistently stored inside NATS JetStream, newly restarted instances were able to reconstruct the complete seat state by replaying historical events during initialization.

After the recovery process, seat availability remained fully consistent with previously completed reservations, demonstrating that no booking information was lost even after multiple service crashes.

The experiment validates both the scalability and resilience properties of the architecture. Stateless request distribution through the gateway enables horizontal scaling, while event replay guarantees durable state reconstruction for stateful services such as the Booking Service.

It is worth noting that both the Catalog Service and the Recommendation Service are fully stateless. As a result, they can be easily scaled horizontally by simply adding or removing instances without requiring state synchronization or recovery mechanisms.

VI. CONCLUSIONS

The developed platform demonstrates how a microservices architecture can support a scalable and resilient cinema reservation system. By combining gRPC synchronous communication with asynchronous event-driven interactions through NATS JetStream, the system achieves consistency, fault tolerance, and responsiveness.

The adoption of patterns such as Circuit Breaker, Event Sourcing, and Strategy Pattern improves reliability and flexibility, while experimental results confirm the system's ability to correctly handle concurrent reservations and service failures in a distributed environment.

A. Limitations

Despite the advantages of the proposed solution, several limitations can be identified. The API Gateway represents a potential single entry point that may become a bottleneck

under very high traffic conditions. However, given its stateless nature and limited responsibilities, it can be easily scaled horizontally to mitigate this issue.

Another limitation is introduced by the use of gRPC for synchronous communication between services. While it provides high performance and strongly typed contracts, it also introduces tighter coupling between services and requires blocking communication patterns. In contrast, the event-driven components of the system are fully decoupled and more flexible, but they cannot fully replace synchronous interactions where immediate responses are required, such as seat reservation requests.

B. Future Improvements

Future improvements of the platform could focus on several key areas. A first enhancement concerns the introduction of a complete user management system, including authentication, authorization, and persistent user profiles, enabling stronger security and personalized recommendation features.

Another relevant extension would be the integration of a payment service to support a fully transactional booking workflow, together with a real email notification system replacing the current simulation-based approach.

The recommendation engine could also be improved by adopting Machine Learning or AI-based techniques to provide more accurate and adaptive suggestions. Similarly, the Catalog Service currently relies on a static `movies.json` file, which limits scalability; future versions could migrate toward persistent storage solutions such as PostgreSQL or MongoDB, potentially combined with Elasticsearch for advanced search and indexing capabilities.

Although Event Sourcing is already used for state reconstruction, the Booking Service maintains its runtime state in memory in order to satisfy the project requirement of implementing a stateful microservice. Likewise, gRPC was adopted as the primary synchronous communication mechanism between services, as it was required by the project specifications. In a production environment, alternative design choices could be considered to improve flexibility and decouple these constraints.

Finally, further improvements could target system observability and architecture refinement, including centralized logging and tracing, as well as a better separation between synchronous gRPC communication and asynchronous event-driven workflows to reduce coupling and improve scalability.

REFERENCES

- [1] gRPC Authors, "gRPC: A high performance, open source universal RPC framework," <https://grpc.io/>, Accessed: 2026.
- [2] NATS Authors, "NATS: Connective Technology for Adaptive Edge and Distributed Systems," <https://nats.io/>, Accessed: 2026.
- [3] Redis Ltd., "Redis Documentation," <https://redis.io/docs/>, Accessed: 2026.
- [4] Sony, "gobreaker: Circuit Breaker pattern implementation for Go," <https://github.com/sony/gobreaker>, Accessed: 2026.
- [5] M. Fowler, "Event Sourcing," <https://martinfowler.com/eaDev/EventSourcing.html>, Accessed: 2026.
- [6] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.